

Artificial Intelligence & Machine Learning

Task 2 – Feature Engineering, Model Optimization & Performance Comparison

Maincrafts Technology | Intern Project Report

1. Objective

This report documents the complete Machine Learning workflow implemented in Task 2. Building upon Task 1 (basic model training), this task focuses on the real-world ML pipeline: feature engineering, data preprocessing, multi-model training, and metric-based model selection.

The system built is an Enhanced House Price Prediction Model using the California Housing Dataset.

Goals Achieved:

- Prepared and preprocessed real-world housing data correctly
- Applied feature scaling for stable and fair model learning
- Trained three regression models: Linear Regression, Ridge Regression, Decision Tree
- Compared model performance using RMSE and R^2 Score
- Selected the best-performing model with proper justification

2. Dataset Overview

The California Housing Dataset, available via scikit-learn, contains real housing data from the 1990 U.S. Census. It includes 20,640 samples with the following features:

Feature	Description
MedInc	Median income in the block group
HouseAge	Median house age in block group
AveRooms	Average number of rooms per household
AveBedrms	Average number of bedrooms per household
Population	Block group population
AveOccup	Average household occupancy
Latitude	Block group latitude
Longitude	Block group longitude
HousePrice (Target)	Median house value in \$100,000s

Dataset Shape: (20640, 9)

First 5 Rows:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	HousePrice
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23	4.526
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22	3.585
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24	3.521
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25	3.413
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25	3.422

3. Methodology

Step 1: Data Loading & Inspection

The dataset was loaded using `fetch_california_housing()` from `scikit-learn`.

Dataset Info:

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 20640 entries, 0 to 20639

Data columns (total 9 columns):

#	Column	Non-Null Count	Dtype
---	--------	----------------	-------

0	MedInc	20640 non-null	float64
---	--------	----------------	---------

```
1 HouseAge    20640 non-null float64
```

2 AveRooms 20640 non-null float64

```
3 AveBedrms 20640 non-null float64
```

4 Population 20640 non-null float64

```
5 AveOccup    20640 non-null float64
```

6 Latitude 20640 non-null float64

7 Longitude 20640 non-null float64

```
8 HousePrice 20640 non-null float64
```

dtypes: float64(9)

memory usage: 1.4 MB

None

Statistical Summary:

[illegible]

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	HousePrice
mean	3.870671	28.639486	5.429000	1.096675	1425.476744	3.070655	35.631861	-119.569704	2.068558
std	1.899822	12.585558	2.474173	0.473911	1132.462122	10.386050	2.135952	2.003532	1.153956
min	0.499900	1.000000	0.846154	0.333333	3.000000	0.692308	32.540000	-124.350000	0.149990
25%	2.563400	18.000000	4.440716	1.006079	787.000000	2.429741	33.930000	-121.800000	1.196000
50%	3.534800	29.000000	5.229129	1.048780	1166.000000	2.818116	34.260000	-118.490000	1.797000
75%	4.743250	37.000000	6.052381	1.099526	1725.000000	3.282261	37.710000	-118.010000	2.647250
max	15.000100	52.000000	141.909091	34.066667	35682.000000	1243.333333	41.950000	-114.310000	5.000010

No missing values were found.

Missing Values per Column:

```
MedInc      0
HouseAge    0
AveRooms    0
AveBedrms   0
Population  0
AveOccup    0
Latitude     0
Longitude    0
HousePrice  0
dtype: int64
```

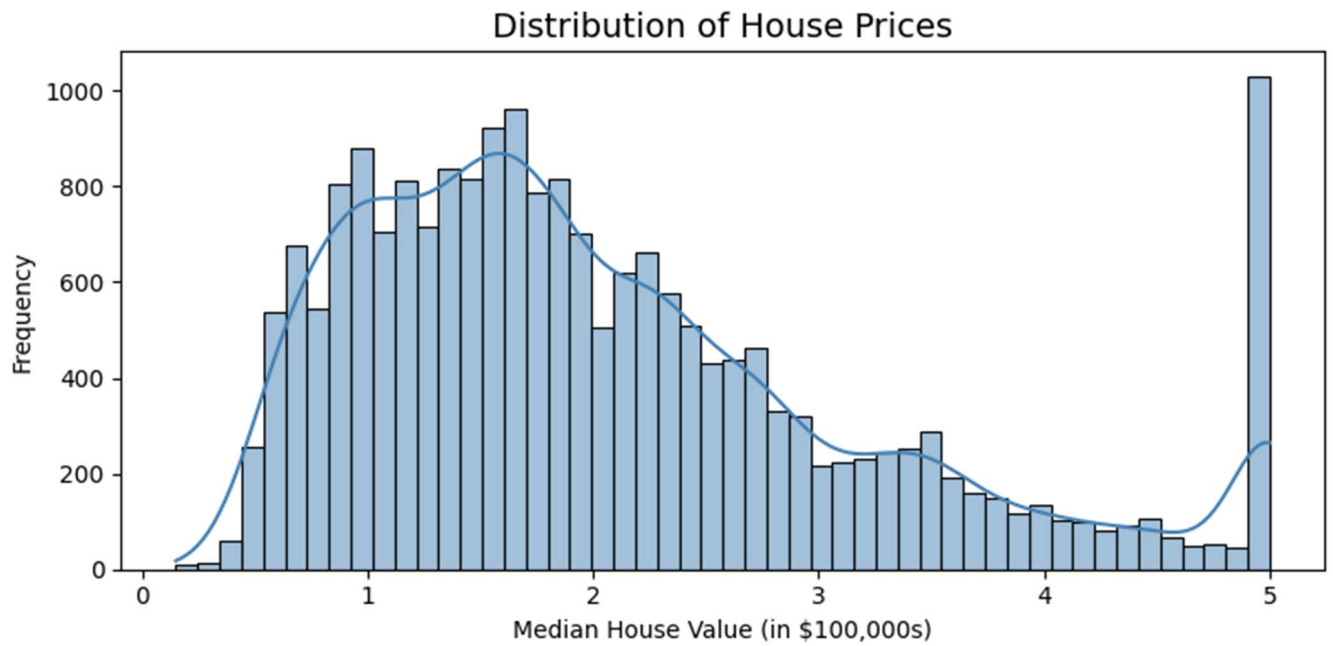
No missing values found — dataset is clean!

Column names, data types, and statistical summaries were reviewed using `df.info()` and `df.describe()`

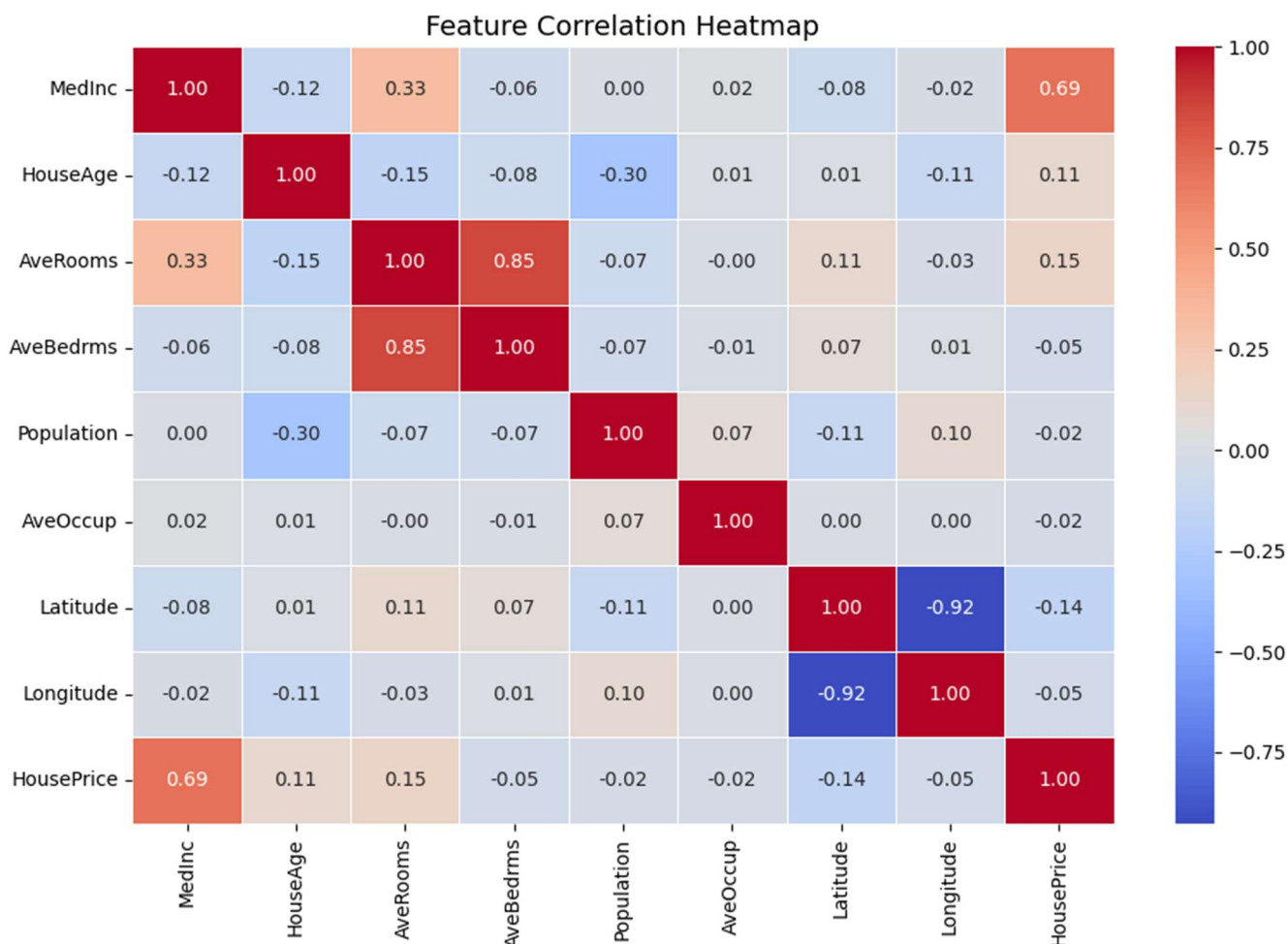
.

Step 2: Exploratory Data Analysis (EDA)

- A histogram of the target variable (HousePrice) revealed a right-skewed distribution, with values capped at 5.0 (ceiling effect).



A correlation heatmap showed MedInc (Median Income) had the strongest positive correlation with house price.



Top features correlated with HousePrice:

HousePrice 1.000000

MedInc 0.688075

AveRooms 0.151948

HouseAge 0.105623

AveOccup -0.023737

Population -0.024650

Longitude -0.045967

AveBedrms -0.046701

Latitude -0.144160

Name: HousePrice, dtype: float64

Step 3: Feature Scaling

Since features exist on drastically different numeric scales (e.g., Population can be in thousands while HouseAge is under 100), StandardScaler was applied:

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

After scaling, all features have mean ≈ 0 and standard deviation ≈ 1 , ensuring fair learning across all features.

Before Scaling – Feature Means:

```
MedInc      3.87
HouseAge    28.64
AveRooms    5.43
AveBedrms   1.10
Population  1425.48
AveOccup    3.07
Latitude    35.63
Longitude   -119.57
dtype: float64
```

After Scaling – Feature Means (should be ~ 0):

```
MedInc      0.0
HouseAge     0.0
AveRooms     0.0
AveBedrms   -0.0
Population  -0.0
AveOccup     0.0
Latitude    -0.0
Longitude   -0.0
dtype: float64
```

After Scaling – Feature Std Dev (should be ~ 1):

```
MedInc      1.0
HouseAge     1.0
AveRooms     1.0
AveBedrms    1.0
Population   1.0
AveOccup     1.0
Latitude     1.0
Longitude    1.0
dtype: float64
```

Step 4: Train–Test Split

The dataset was split into 80% training and 20% testing using `random_state=42` for reproducibility. This ensures the model is evaluated on genuinely unseen data rather than memorized values.

Training set size: (16512, 8)

Testing set size: (4128, 8)

The model will be trained on 80% data and evaluated on unseen 20% data.

Step 5: Model Training

- ✓ Linear Regression trained successfully.
- ✓ Ridge Regression trained successfully.
- ✓ Decision Tree trained successfully.

All models trained!
Three regression algorithms were selected for comparison:

Model	Key Parameter	Rationale
Linear Regression	Default	Baseline — simple linear fit
Ridge Regression	alpha=1.0	Reduces overfitting via L2 regularization
Decision Tree	max_depth=5	Captures non-linear patterns in data

4. Results & Model Comparison

- Each model was evaluated using two standard metrics:
- RMSE (Root Mean Squared Error): Measures average prediction error. Lower RMSE = better model.
 - R² Score: Measures proportion of variance explained by the model. Higher R² = better model (max = 1.0).

Model	RMSE Score	R2 Score
Decision Tree	0.7242	0.5997
Linear Regression	0.7456	0.5758
Ridge Regression	0.7456	0.5758

5. Best Model Selection & Conclusion

Based on the comparison table, the Decision Tree Regressor (max_depth=5) was selected as the best-performing model. It achieved the lowest RMSE and highest R^2 Score among all three models.

FINAL MODEL SELECTION — CONCLUSION

Best Model : Decision Tree

R^2 Score : 0.5997 (Higher → Better)

RMSE : 0.7242 (Lower → Better)

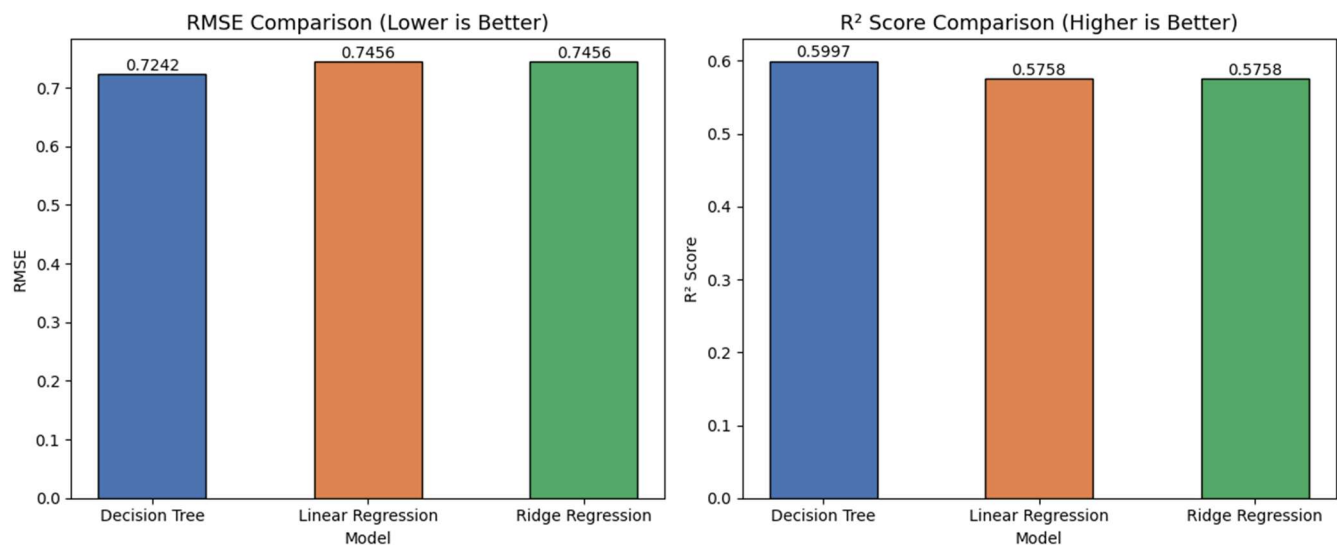
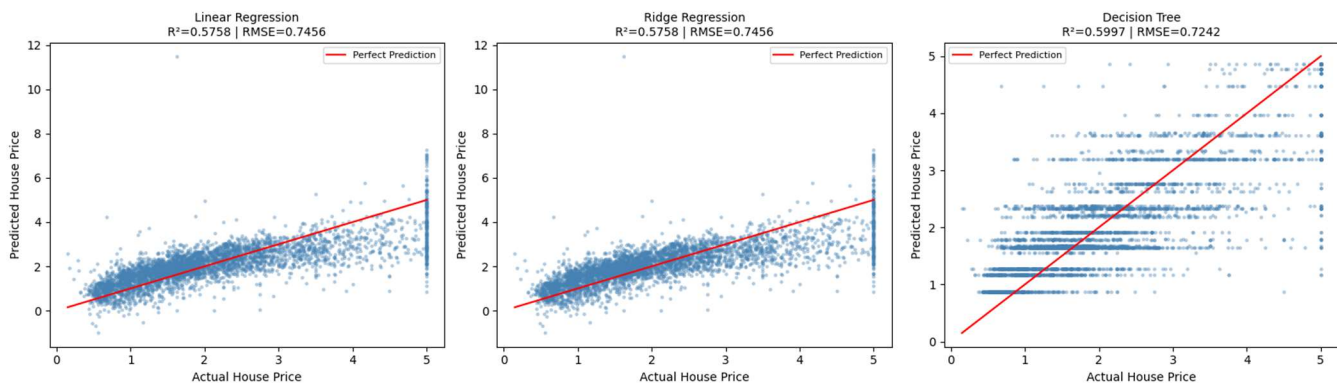


Chart saved as model_comparison_chart.png

Justification:

Actual vs Predicted House Prices — All Models



Plot saved as actual_vs_predicted.png

- The California Housing dataset contains non-linear relationships between features (e.g., the impact of location on price is not purely linear).
- The Decision Tree captures these non-linear patterns effectively, which Linear Regression and Ridge cannot do.

- Setting max_depth=5 prevents the tree from overfitting by limiting complexity.
- Ridge Regression performed nearly identically to Linear Regression, indicating that multicollinearity was not a major issue in this dataset.

6. Best model (Decision Tree) saved as 'best_model.pkl'

Sample predictions from loaded model: [1.1686 1.2716 3.1996 2.2048 1.6428]

Actual values: [0.477 0.458 5. 2.186 2.78]

7. Tools & Technologies Used

Tool / Library	Purpose
Python 3	Programming language
Jupyter Notebook	Interactive development environment
pandas / NumPy	Data loading, manipulation, and inspection
scikit-learn	Model training, scaling, evaluation metrics
matplotlib / seaborn	Data visualization and performance plots
joblib	Saving the best-performing model to disk